LAB 7 : From Neurons to Vision Models

What You Already Know

So far, everything you built can be summarized as:

- A neuron computes a score.
- An activation changes that score into a usable signal.
- A loss measures how wrong the output is.
- Backpropagation computes how each parameter contributed to the error.
- An update rule changes parameters to reduce loss.

A single neuron:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Activation:

$$a = \phi(z)$$

Loss:

$$L = L(y, \hat{y})$$

Update idea:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

This logic does not depend on images.

It applies to any input: numbers, signals, text features, sensor readings, etc.

2. What Is a Fully Connected Layer?

A layer is called **fully connected (dense)** when:

every output neuron uses every input value.

If the input has nnn numbers and the next layer has mmm neurons:

- each neuron needs nnn weights
- each neuron has 1 bias
- total parameters = weights + biases

Weights:

$$n \times m$$

Biases:

$$m$$

Total:

$$(n \times m) + m$$

Example: input has 4 numbers, next layer has 3 neurons:

$$(4 \times 3) + 3 = 15$$

This is the type of network you have already built.

3. Do We Need Images to Talk About CNNs?

No.

CNN ideas (local patterns, sliding filters, parameter sharing) are not “only for images”.

They can be used for:

- 1D signals (EEG, ECG, audio)
- time series (sensor sequences)

- text as sequences (characters/words)
- images (2D)

But images are used first because the ideas are easiest to see visually:

- “local region”
- “nearby pixels”
- “sliding window”
- “edges and corners”

So we will start with images for intuition, but remember:

CNN is a structural idea, not an image-only trick.

4. Why Fully Connected Layers Become a Problem for Images

A fully connected layer ignores structure.

It treats an image like a long list of numbers.

Example: a small grayscale image 28×28

Number of inputs:

$$28 \times 28 = 784$$

If the first hidden layer has 128 neurons, weights are:

$$784 \times 128 = 100352$$

That is only the first layer.

Now the deeper point:

In images, nearby pixels are related.

A dense layer does not use this fact.

It connects:

- a pixel in the top-left corner

- to a neuron
- with the same “importance style” as any other pixel

So the dense layer wastes parameters.

5. What CNN Changes: The One Key Idea

CNN replaces:

“use all inputs everywhere”

with:

“use small local regions repeatedly”.

Instead of each neuron looking at 784 pixels, we look at a small patch.

Example: a 3×3 patch has:

$$3 \times 3 = 9$$

So instead of learning hundreds of weights per neuron, we learn just 9 weights per filter.

That small weight grid is called:

Filter (Kernel)

6. What Is Convolution?

Convolution is a repeated local weighted sum.

You:

1. take a small patch of the input
2. multiply it elementwise with the filter
3. sum the result to get one output number
4. slide the filter and repeat

Mathematical form (2D):

$$\text{out}(i,j) = \sum_{m=0}^{f-1} \sum_{n=0}^{f-1} \text{in}(i+m, j+n) \cdot K(m,n)$$

Conceptually:

It is the same neuron computation:

$$z = \sum w_i x_i + b$$

but now:

- the xi come from a local patch
- the same wi are reused at every location

That reuse is the major change.

7. Why Reusing the Same Filter Matters

In dense layers:

- every neuron has its own separate weights

In convolution:

- the same filter weights are used at many positions

This is called:

Parameter sharing

That is why CNNs scale.

Instead of learning a new edge detector for every pixel location, CNN learns one edge detector and applies it everywhere.

8. What Is a Feature Map?

When you slide a filter across the input, you produce a grid of outputs.

That grid is called a:

Feature map

Each filter produces one feature map.

If you use 16 filters, you get 16 feature maps.

So the output of a convolution layer is not one number, it is a stack of learned maps.

---

9. Why Activation Still Matters After Convolution

Without activation, stacking convolutions still becomes a linear system.

So after convolution, we apply activation:

$$Z = \mathrm{Conv}(X, K) + b$$

$$A = \phi(Z)$$

Common choice is ReLU:

$$\mathrm{ReLU}(z) = \max(0, z)$$

Reason (connected to DLQ6):

Sigmoid slopes are usually less than 1, so gradients shrink over depth.

ReLU (when active) has slope 1, so it does not automatically shrink gradients.

10. Padding: Why Output Size Changes, and How We Control It

If you apply a 3×3 filter on an image without padding, the output shrinks.

If input is N×N and filter is F×F, stride S, padding P:

Output size:

$$\mathrm{out} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Padding means adding zeros around the border.

Why padding is used:

- to keep spatial size from shrinking too fast

- to allow edge pixels to be treated fairly
- to control output dimension

Common special case (stride 1):

To keep output same size as input with a 3×3 filter:

$$P = 1$$

11. Pooling: What It Does and Why It Exists

Pooling reduces spatial size by summarizing small regions.

Max pooling takes the maximum in each region.

Example: 2×2 max pool:

$$\text{pool} = \max(\text{region})$$

Why pooling is used:

- reduces computation
- reduces number of parameters in later layers
- keeps strongest signal
- adds tolerance to small shifts (small translation invariance)

Pooling is not learning.

It has no weights.

12. Dropout: Why Networks Overfit

In large networks, units can become dependent on each other.

Dropout randomly zeros some activations during training:

$$A' = A \cdot M$$

Where M is a random mask of 0s and 1s.

Why dropout works:

- forces the network to distribute information
- prevents single “dominant neurons”
- reduces overfitting

Dropout is only used during training.

13. Optimizers: Why Basic Gradient Descent Isn’t Enough

Basic gradient descent update:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

This is called SGD (conceptually).

Deep networks often need more stable updates because:

- gradients vary by layer
- loss surface is not smooth and can oscillate
- learning can become slow or unstable

Optimizers modify how we step:

- Momentum: smooths oscillations
- RMSProp: adapts learning rate per parameter
- Adam: combines momentum + adaptive scaling

Optimizers do not change the model architecture.

They change how learning moves.

14. Separation of Roles: Do Not Mix These

Activation is not optimizer.

Loss is not gradient.

Gradient is not regularization.

A clean separation:

- Activation: changes representation during forward pass
- Loss: measures error after forward pass
- Gradient: computes direction during backward pass
- Optimizer: decides the update rule after gradients exist
- Regularization: controls overfitting during training

15. How This Connects Back to DLQ6

DLQ6 showed:

- gradients multiply across layers
- depth can make learning fragile
- sigmoid deep networks shrink gradients fast
- width does not fix vanishing by itself

LAB 7 adds the next realization:

The model does not only fail because of “math”.

It fails because of structure.

Dense layers ignore structure in images.

CNN fixes this by:

- local connectivity
- parameter sharing
- hierarchical feature learning

And then we still need:

- activations for nonlinearity
- optimizers for stable learning
- regularization for generalization

Deep learning is a system.

16. What You Should Be Able to Say After This Lab

You should be able to explain:

- what a dense layer is and why it scales badly for images
- why CNN uses local regions and reused filters
- what a filter is and what a feature map is
- why we still need activation after convolution
- what padding controls and why output size changes
- what pooling does and why it is useful
- what dropout is and why it reduces overfitting
- why optimizers exist and what they improve
- why DLQ6 gradient fragility matters in CNNs too

> Until now, we learned how learning updates numbers.
>
> Now we learn that architecture decides whether those updates can survive long enough to matter.
>
> If signals cannot travel, learning cannot happen.